



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE N° 02

NOMBRE COMPLETO: GARCÍA SITIO DIEGO

N° de Cuenta: 318120601

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 05/09/2026

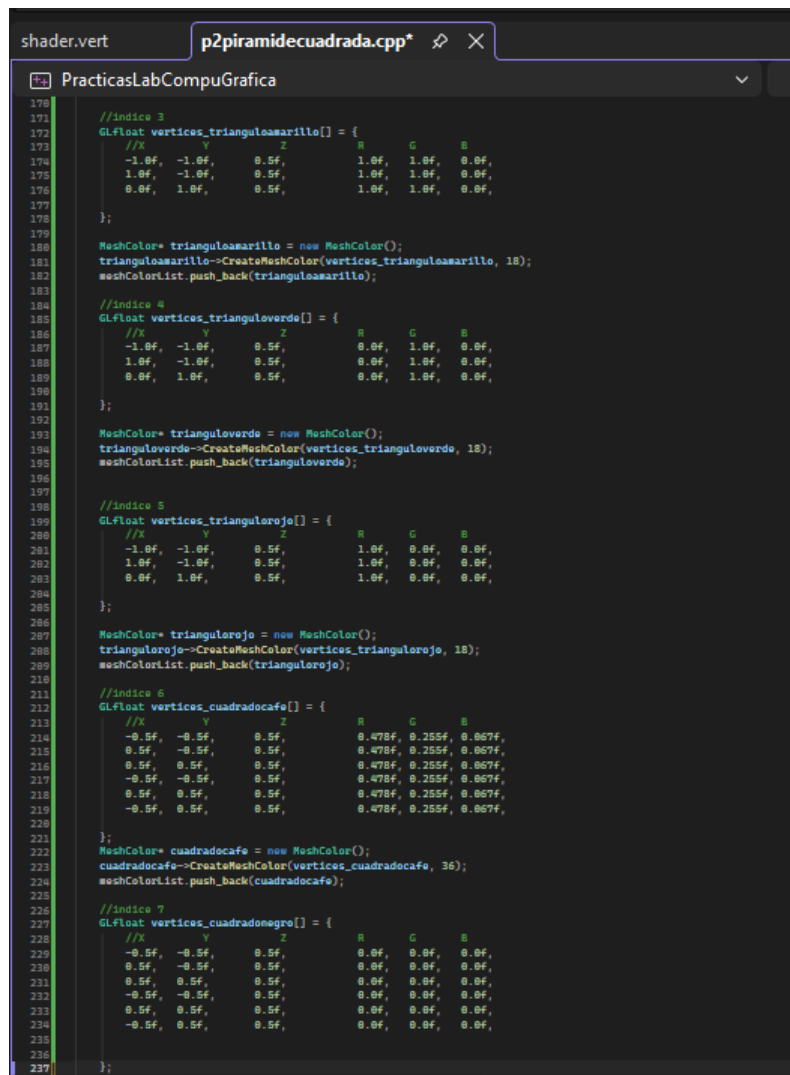
CALIFICACIÓN: _____

EJERCICIO 1

Descripción: En este ejercicio se debe generar las figuras copiando los vértices de triangulomagenta y cuadradoazul:

- triángulo amarillo
- triángulo verde
- triángulo rojo
- cuadrado café (0.478, 0.255, 0.067)

Solución: Creamos los triángulos y los cuadrados de los colores requeridos, imitando el código que el profesor nos proporcionó, todo esto lo vemos en la ilustración 1 (también está el cuadrado negro que se usará en el segundo ejercicio).



```
shader.vert  p2piramidecuadrada.cpp* X
PracticasLabCompuGrafica
170
171 //indice 3
172 GLfloat vertices_trianguloamarillo[] = {
173     //X      Y      Z      R      G      B
174     -1.0f, -1.0f,  0.5f,  1.0f,  1.0f,  0.0f,
175     1.0f, -1.0f,  0.5f,  1.0f,  1.0f,  0.0f,
176     0.0f,  1.0f,  0.5f,  1.0f,  1.0f,  0.0f,
177 };
178
179 MeshColor* trianguloamarillo = new MeshColor();
180 trianguloamarillo->CreateMeshColor(vertices_trianguloamarillo, 18);
181 meshColorList.push_back(trianguloamarillo);
182
183 //indice 4
184 GLfloat vertices_trianguloverde[] = {
185     //X      Y      Z      R      G      B
186     -1.0f, -1.0f,  0.5f,  0.0f,  1.0f,  0.0f,
187     1.0f, -1.0f,  0.5f,  0.0f,  1.0f,  0.0f,
188     0.0f,  1.0f,  0.5f,  0.0f,  1.0f,  0.0f,
189 };
190
191 MeshColor* trianguloverde = new MeshColor();
192 trianguloverde->CreateMeshColor(vertices_trianguloverde, 18);
193 meshColorList.push_back(trianguloverde);
194
195 //indice 5
196 GLfloat vertices_triangulorojo[] = {
197     //X      Y      Z      R      G      B
198     -1.0f, -1.0f,  0.5f,  1.0f,  0.0f,  0.0f,
199     1.0f, -1.0f,  0.5f,  1.0f,  0.0f,  0.0f,
200     0.0f,  1.0f,  0.5f,  1.0f,  0.0f,  0.0f,
201 };
202
203 MeshColor* triangulorojo = new MeshColor();
204 triangulorojo->CreateMeshColor(vertices_triangulorojo, 18);
205 meshColorList.push_back(triangulorojo);
206
207 //indice 6
208 GLfloat vertices_cuadradocafe[] = {
209     //X      Y      Z      R      G      B
210     -0.5f, -0.5f,  0.5f,  0.478f, 0.255f, 0.067f,
211     0.5f, -0.5f,  0.5f,  0.478f, 0.255f, 0.067f,
212     0.5f,  0.5f,  0.5f,  0.478f, 0.255f, 0.067f,
213     -0.5f, -0.5f,  0.5f,  0.478f, 0.255f, 0.067f,
214     0.5f,  0.5f,  0.5f,  0.478f, 0.255f, 0.067f,
215     -0.5f, 0.5f,  0.5f,  0.478f, 0.255f, 0.067f,
216 };
217
218 MeshColor* cuadradocafe = new MeshColor();
219 cuadradocafe->CreateMeshColor(vertices_cuadradocafe, 36);
220 meshColorList.push_back(cuadradocafe);
221
222 //indice 7
223 GLfloat vertices_cuadradonegro[] = {
224     //X      Y      Z      R      G      B
225     -0.5f, -0.5f,  0.5f,  0.0f,  0.0f,  0.0f,
226     0.5f, -0.5f,  0.5f,  0.0f,  0.0f,  0.0f,
227     0.5f,  0.5f,  0.5f,  0.0f,  0.0f,  0.0f,
228     -0.5f, -0.5f,  0.5f,  0.0f,  0.0f,  0.0f,
229     0.5f,  0.5f,  0.5f,  0.0f,  0.0f,  0.0f,
230     -0.5f, 0.5f,  0.5f,  0.0f,  0.0f,  0.0f,
231 };
232
233 MeshColor* cuadradonegro = new MeshColor();
234 cuadradonegro->CreateMeshColor(vertices_cuadradonegro, 36);
235 meshColorList.push_back(cuadradonegro);
236
237 }
```

Ilustración 1: Código para los colores de las figuras del primer ejercicio.

EJERCICIO 2

Descripción: En este ejercicio se debe usar la proyección ortogonal para generar el siguiente dibujo a partir de instancias de las figuras anteriormente creadas, recordar que todos se dibujan en el origen y por transformaciones geométricas se desplazan. Los rombos azul y café son cuadrados rotados sobre el eje Z.

Color diferente a blanco, negro y los de las figuras

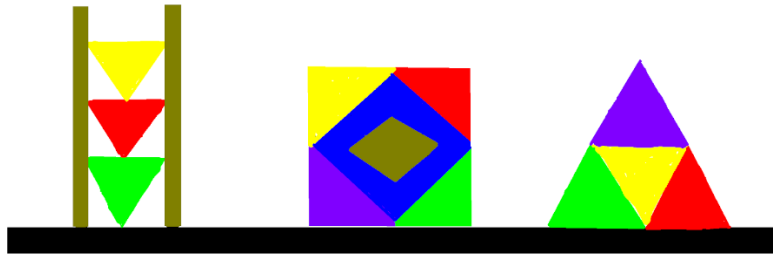


Ilustración 2: Imagen de lo que se debe conseguir en el ejercicio 2.

Solución: Antes de empezar a colocar el código para las figuras geométricas, debemos colocar las siguientes líneas de código de la ilustración 3, ya que nos permitirá usar la proyección ortogonal requerida por el ejercicio.

```
243 //Projection: Matriz de Dimensión 4x4 para indicar si vemos en 2D( orthogonal) o en 3D) perspectiva
244 glm::mat4 projection = glm::ortho(-1.0f, 1.0f, -1.0f, 1.0f, 0.1f, 100.0f); //cuántas unidades voy a lograr a ver, eso es la variable final de la función
245 //
246 //La proyección ortogonal no toma en cuenta la profundidad
247 //
248 // (filoview se comporta como un ojo humano - angulo del ojo dilatado, aspect ratio, z mid?, z fag?)
249 glm::mat4 projection = glm::perspective(glm::radians(60.0f), mainWindow.getBufferWidth() / mainWindow.getBufferHeight(), 0.1f, 100.0f);
250 //
251 //Model: Matriz de Dimensión 4x4 en la cual se almacena la multiplicación de las transformaciones geométricas.
252 glm::mat4 model(1.0); //fuera del while se usa para inicializar la matriz con una identidad
253 //
254 //Loop mientras no se cierra la ventana
255 while (!mainWindow.getShouldClose())
```

Ilustración 3: Líneas de código necesarias antes de empezar el While.

Ahora, colocaremos el código necesario para crear las 4 figuras de la ilustración 2, además del color de fondo que es una especie de color anaranjado muy claro que se observa en la ilustración 4:

1. El piso negro, ilustración 5
2. La torre de lanzamiento de cohetes, ilustración 6
3. El holocrón, ilustración 7
4. Pirámide cuadrangular (no recuerdo si el profesor dijo exactamente que así se llamaba la figura) , ilustración 8

Y con esto, tenemos la ejecución del programa en la ilustración 9.

```

281 //Loop mientras no se cierra la ventana
282 while (!mainWindow.getShouldClose())
283 {
284     //Recibir eventos del usuario
285     glfwPollEvents();
286     //Limpiar la ventana
287     // Fondo gris claro
288     //glClearColor(0.9f, 0.9f, 0.9f, 1.0f);
289     glClearColor(1.0f, 0.9f, 0.8f, 1.0f); //naranja ultraduper claro
290
291     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT); //Se agrega limpiar el buffer de profundidad
292
293     //Para el cubo y las pirámides se usa el primer set de shaders con índice 0 en ShaderList
294     // Activamos el shader que lee el color
295     shaderList[1].useShader();
296     uniformModel = shaderList[1].getModelLocation();
297     uniformProjection = shaderList[1].getProjectLocation();
298     /*
299     angulo += 0.01;
300     */
301

```

Ilustración 4: Código para iniciar a colocar las figuras junto con el fondo anaranjado.

```

304 //PISO NEGRO
305 model = glm::mat4(1.0);
306 model = glm::translate(model, glm::vec3(0.0f, -0.8f, -3.0f));
307 model = glm::scale(model, glm::vec3(2.0f, 0.1f, 1.0f));
308 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
309 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
310 meshColorList[7] -> RenderMeshColor();
311
312
313

```

Ilustración 5: Código para hacer el piso negro.

```

316 // TORRE DE LANZAMIENTO DE COHETES
317 // Cubo Izquierdo
318 model = glm::mat4(1.0);
319 model = glm::translate(model, glm::vec3(-0.82f, -0.35f, -3.0f));
320 model = glm::scale(model, glm::vec3(0.06f, 0.8f, 1.0f));
321 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
322 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
323 meshColorList[6]->RenderMeshColor();
324 // Cubo Derecho
325 model = glm::mat4(1.0);
326 model = glm::translate(model, glm::vec3(-0.48f, -0.35f, -3.0f));
327 model = glm::scale(model, glm::vec3(0.06f, 0.8f, 1.0f));
328 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
329 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
330 meshColorList[6]->RenderMeshColor();
331 // Piramide Verde Invertido
332 model = glm::mat4(1.0);
333 model = glm::translate(model, glm::vec3(-0.65f, -0.63f, -3.0f));
334 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
335 model = glm::scale(model, glm::vec3(0.14f, 0.12f, 1.0f));
336 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
337 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
338 meshColorList[4]->RenderMeshColor();
339 // Piramide Roja Invertido
340 model = glm::mat4(1.0);
341 model = glm::translate(model, glm::vec3(-0.65f, -0.39f, -3.0f));
342 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
343 model = glm::scale(model, glm::vec3(0.14f, 0.12f, 1.0f));
344 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
345 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
346 meshColorList[5]->RenderMeshColor();
347 // Piramide Amarilla Invertido
348 model = glm::mat4(1.0);
349 model = glm::translate(model, glm::vec3(-0.65f, -0.15f, -3.0f));
350 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
351 model = glm::scale(model, glm::vec3(0.14f, 0.12f, 1.0f));
352 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
353 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
354 meshColorList[3]->RenderMeshColor();

```

Ilustración 6: Código para la torre de lanzamiento de cohetes.

```

360 // HOLOCRON
361 //Para que las esquinas sean rectas, usamos un Scale asimétrico (X=0.176, Y=0.088).
362 // Esto achata el triángulo, convirtiendo su punta en un ángulo perfecto de 90 grados.
363
364 //Con variables porque agiliza moverlo ya que es el que más se me complico
365 float s = 0.25f; // Distancia desde el centro a las esquinas
366 float sy = -0.50f; // Altura central de la figura
367 //
368 // Piramide Amarilla
369 model = glm::mat4(1.0);
370 model = glm::translate(model, glm::vec3(-s, sy + s, -3.0f)); // Movemos a la esquina top-left
371 model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f)); // Rotamos
372 model = glm::translate(model, glm::vec3(0.0f, -0.088f, 0.0f)); // Bajamos la punta al centro de rotación
373 model = glm::scale(model, glm::vec3(0.176f, 0.088f, 1.0f)); // El scale mágico
374 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
375 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
376 meshColorList[3]-->RenderMeshColor();
377 // Piramide Roja
378 model = glm::mat4(1.0);
379 model = glm::translate(model, glm::vec3(s, sy + s, -3.0f));
380 model = glm::rotate(model, glm::radians(315.0f), glm::vec3(0.0f, 0.0f, 1.0f));
381 model = glm::translate(model, glm::vec3(0.0f, -0.088f, 0.0f));
382 model = glm::scale(model, glm::vec3(0.176f, 0.088f, 1.0f));
383 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
384 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
385 meshColorList[5]-->RenderMeshColor();
386 // Piramide Magenta
387 model = glm::mat4(1.0);
388 model = glm::translate(model, glm::vec3(-s, sy - s, -3.0f));
389 model = glm::rotate(model, glm::radians(135.0f), glm::vec3(0.0f, 0.0f, 1.0f));
390 model = glm::translate(model, glm::vec3(0.0f, -0.088f, 0.0f));
391 model = glm::scale(model, glm::vec3(0.176f, 0.088f, 1.0f));
392 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
393 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
394 meshColorList[1]-->RenderMeshColor();
395 // Piramide Verde
396 model = glm::mat4(1.0);
397 model = glm::translate(model, glm::vec3(s, sy - s, -3.0f));
398 model = glm::rotate(model, glm::radians(225.0f), glm::vec3(0.0f, 0.0f, 1.0f));
399 model = glm::translate(model, glm::vec3(0.0f, -0.088f, 0.0f));
400 model = glm::scale(model, glm::vec3(0.176f, 0.088f, 1.0f));
401 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
402 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
403 meshColorList[4]-->RenderMeshColor();
404 // Cubo Azul
405 model = glm::mat4(1.0);
406 model = glm::translate(model, glm::vec3(0.0f, sy, -3.0f));
407 model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
408 model = glm::scale(model, glm::vec3(0.355f, 0.355f, 1.0f)); // Tamaño justo para cubrir la costura
409 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
410 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
411 meshColorList[2]-->RenderMeshColor();
412 // Cubo Café
413 model = glm::mat4(1.0);
414 model = glm::translate(model, glm::vec3(0.0f, sy, -2.0f));
415 model = glm::rotate(model, glm::radians(45.0f), glm::vec3(0.0f, 0.0f, 1.0f));
416 model = glm::scale(model, glm::vec3(0.18f, 0.18f, 1.0f));
417 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
418 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
419 meshColorList[6]-->RenderMeshColor();
420

```

Ilustración 7: Código para El holocrón.

```

424
425 // PIRAMIDE CUADRANGULAR (CREO)
426 // Piramide Verde
427 model = glm::mat4(1.0);
428 model = glm::translate(model, glm::vec3(0.50f, -0.60f, -3.0f));
429 model = glm::scale(model, glm::vec3(0.15f, 0.15f, 1.0f));
430 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
431 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
432 meshColorList[4] -> RenderMeshColor();
433 // Piramide Roja
434 model = glm::mat4(1.0);
435 model = glm::translate(model, glm::vec3(0.80f, -0.60f, -3.0f));
436 model = glm::scale(model, glm::vec3(0.15f, 0.15f, 1.0f));
437 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
438 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
439 meshColorList[5] -> RenderMeshColor();
440 // Piramide Magenta
441 model = glm::mat4(1.0);
442 model = glm::translate(model, glm::vec3(0.65f, -0.30f, -3.0f));
443 model = glm::scale(model, glm::vec3(0.15f, 0.15f, 1.0f));
444 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
445 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
446 meshColorList[1] -> RenderMeshColor();
447 // Piramide Amarilla Invertido
448 model = glm::mat4(1.0);
449 model = glm::translate(model, glm::vec3(0.65f, -0.60f, -3.0f));
450 model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
451 model = glm::scale(model, glm::vec3(0.15f, 0.15f, 1.0f));
452 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
453 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
454 meshColorList[3] -> RenderMeshColor();
455

```

Ilustración 8: Código para la pirámide cuadrangular.

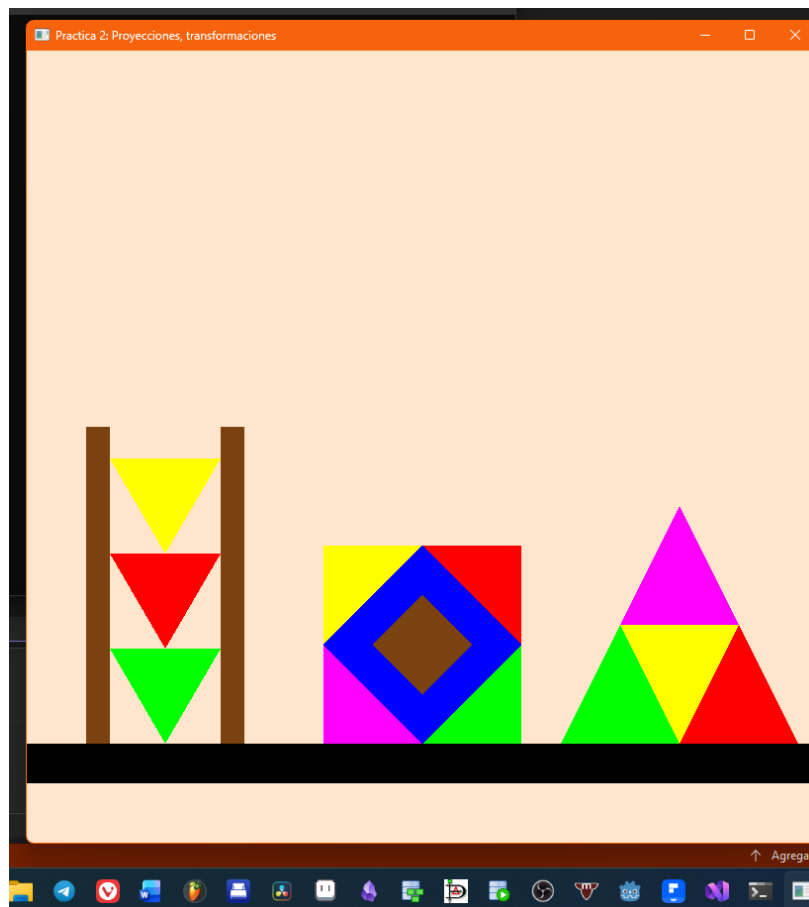


Ilustración 9: Ejecución del ejercicio 2.

CONCLUSIÓN

El mayor reto del ejercicio que tuve se presentó al construir la el holocrón, ya que requería formar ángulos rectos utilizando triángulos que originalmente no tenían esa propiedad geométrica (y porque fue la primera figura que hice).

Para resolverlo, fue necesario aplicar un escalado asimétrico (modificando la proporción en el eje X y Y) que permitiera "aplastar" el triángulo hasta forzar un ángulo exacto de 90 grados en su vértice inferior, cosas que tuve que preguntar porque ya me estaba estresando un poco. Posteriormente, utilicé una traslación interna para alinear el pivote y rotaciones de 45, 135, 225 y 315 grados para colocar cada esquina en su posición final.

Fuera del holocrón, pude hacer las demás figuras haciendo un poco de ensayo y error, creo que después de hacer figuras planas a cuerpos geométricos, el salto de dificultad estuvo fuerte.